

Search ...



Menu

[Home](#) » [Linux](#) » [Linux Commands](#)

Expect command and how to automate shell scripts like magic

👤 Mokhtar Ebrahim Last Updated On: August 6, 2023

In the previous post, we talked about writing [practical shell scripts](#), and we saw how it is easy to write a shell script. Today we are going to talk about a tool that does magic to our shell scripts; that tool is the **Expect command** or **Expect scripting language**.

Expect command or expect scripting language is a language that talks with your interactive programs or scripts that require user interaction.

Expect scripting language works by expecting input, then the Expect script will send the response without any user interaction.

You can say that this tool is your robot, which will automate your scripts.

If the expect command is not installed on your system, you can install it using the following command:

```
$ apt-get install expect
```

Or on Red Hat based systems like CentOS:

```
$ yum install expect
```

Table of Contents [[hide](#)]

- 1 [Expect Command](#)
- 2 [Automate interactive commands](#)
- 3 [Using autoexpect](#)
- 4 [Working with Variables](#)
- 5 [Conditional Tests](#)
- 6 [If else Conditions](#)
- 7 [While Loops](#)
- 8 [For Loops](#)
- 9 [User-defined Functions](#)
- 10 [Interact Command](#)

Expect Command

Before we talk about expect command, Let's see some of the Expect command which used for interaction:

spawn	Starts a script or a program.
expect	Waits for program output.
send	Sends a reply to your program.
interact	Allows you to interact with your program.

- The spawn command starts a script or a program like the shell, [FTP](#), Telnet, SSH, SCP, and so on.
- The send command sends a reply to a script or a program.
- The Expect command waits for input.
- The interact command allows you to define a predefined user interaction.

We are going to type a shell script that asks some questions, and we will make an Expect script that will answer those questions.

First, the shell script will look like this:

```
#!/bin/bash

echo "Hello, who are you?"

read $REPLY

echo "Can I ask you some questions?"

read $REPLY

echo "What is your favorite topic?"

read $REPLY
```

Now we will write the Expect scripts that will answer this automatically:

```
#!/usr/bin/expect -f

set timeout -1

spawn ./questions
```

```
expect "Hello, who are you?\r"

send -- "Im Adam\r"

expect "Can I ask you some questions?\r"

send -- "Sure\r"

expect "What is your favorite topic?\r"

send -- "Technology\r"

expect eof
```

The first line defines the expect command path which is

```
#!/usr/bin/expect
```

On the second line of code, we disable the **timeout**. Then start our script using **spawn** command.

We can use **spawn** to run any program we want or any other interactive script.

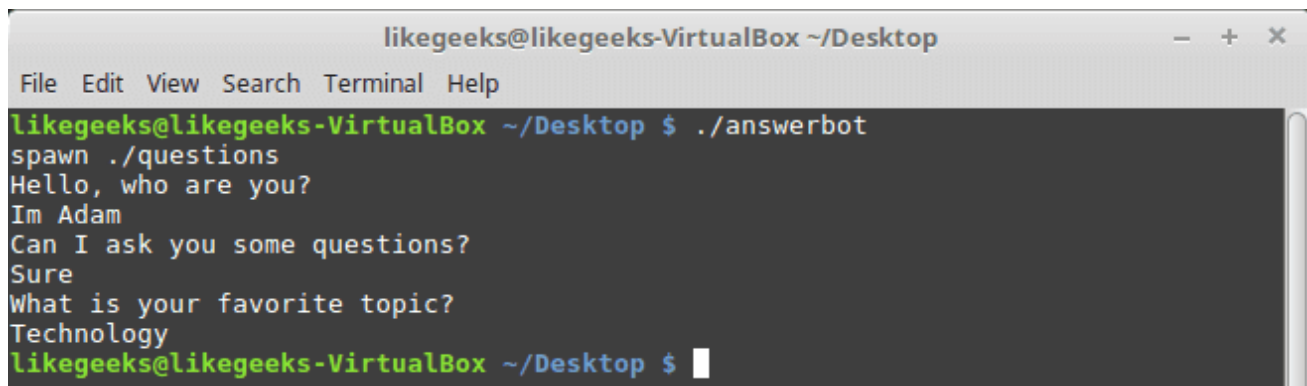
The remaining lines are the Expect script that interacts with our shell script.

The last line means the end of the interaction.

Now Showtime, let's run our answer bot and make sure you make it executable.

```
$ chmod +x ./answerbot
```

```
$ ./answerbot
```

A screenshot of a terminal window titled 'likegeeks@likegeeks-VirtualBox ~/Desktop'. The terminal shows the execution of the './answerbot' script. The output of the script is as follows: 'spawn ./questions', 'Hello, who are you?', 'Im Adam', 'Can I ask you some questions?', 'Sure', 'What is your favorite topic?', 'Technology'. The prompt returns to the shell as 'likegeeks@likegeeks-VirtualBox ~/Desktop \$' with a cursor.

```
likegeeks@likegeeks-VirtualBox ~/Desktop
File Edit View Search Terminal Help
likegeeks@likegeeks-VirtualBox ~/Desktop $ ./answerbot
spawn ./questions
Hello, who are you?
Im Adam
Can I ask you some questions?
Sure
What is your favorite topic?
Technology
likegeeks@likegeeks-VirtualBox ~/Desktop $
```

Cool!! The send command answered all questions.

If you get errors about the location of Expect command you can get the location using the which command:

```
$ which expect
```

We did not interact with our script at all; the Expect program does the job for us.

You can apply the above method to any interactive script or program. Although the above Expect script is very easy to write, maybe the Expect script little tricky for some people, well you have it.

Automate interactive commands

The `-c` option for `expect` allows you to provide `expect` commands directly from the command line rather than from a file.

Let's say you want to spawn a shell, wait for the shell prompt, and then send the command `whoami` :

```
expect -c 'spawn /bin/bash; expect "$ "; send "whoami\n"
```

First, `spawn /bin/bash` initiates a new shell instance.

Then, the `expect "$ "` waits for the shell to present its typical prompt, which is often represented as `$` for a regular user.

Once the script identifies this prompt, `send "whoami\n"` instructs the script to input the `whoami` command into the shell, followed by a newline (represented by `\n`), which is equivalent to pressing the "Enter" key.

The result is the same as if you manually typed the `whoami` command into a shell to find out the name of the currently logged-in user.

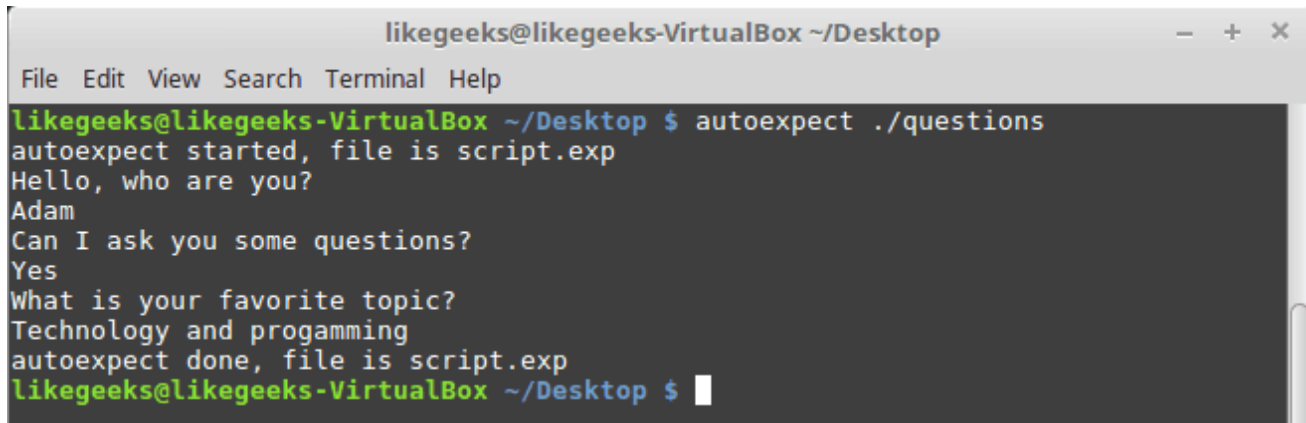
Using autoexpect

To build an expect script automatically, you can use the `autoexpect` command.

`autoexpect` works like `expect` command, but it builds the automation script for you. You pass the script you want to automate to `autoexpect` as a

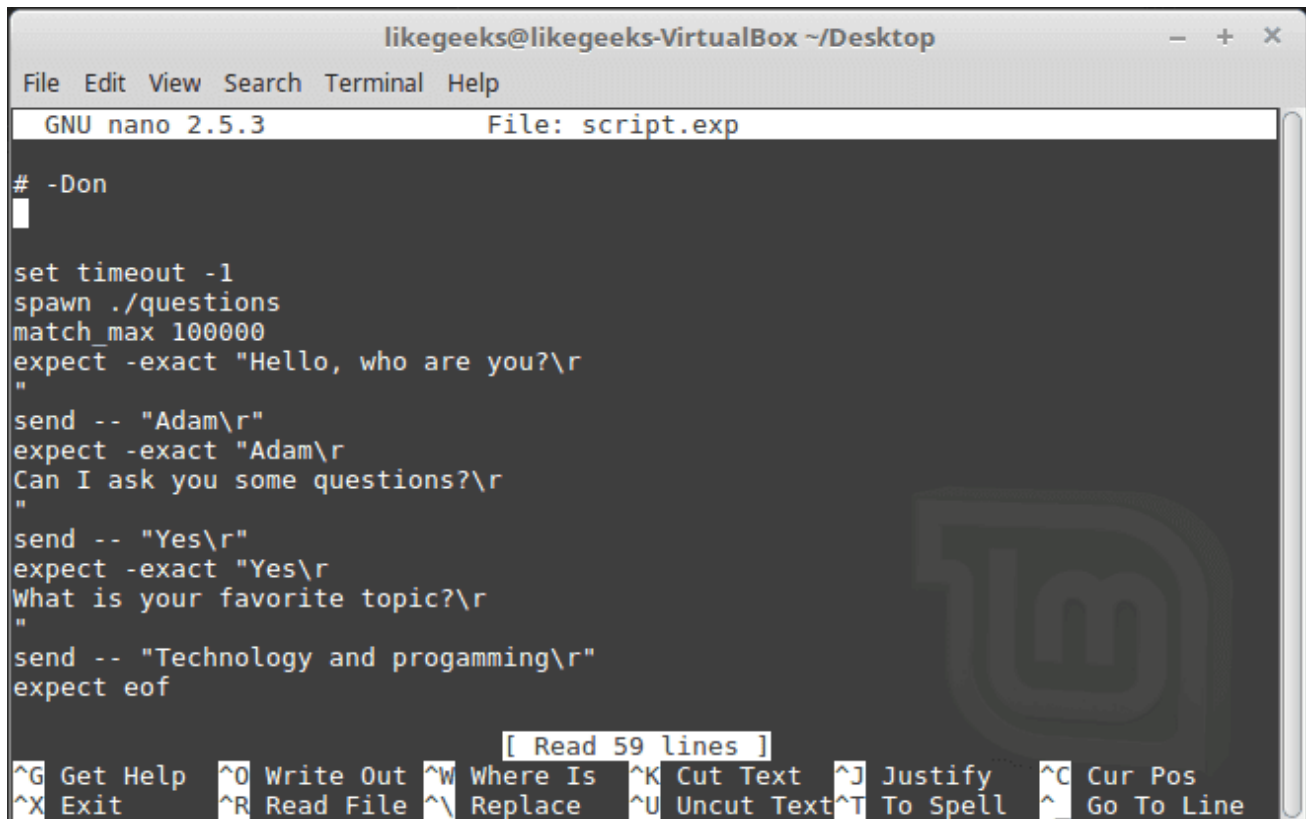
parameter, and you answer the questions, and autoexpect saves your answers in a file.

```
$ autoexpect ./questions
```

A terminal window titled 'likegeeks@likegeeks-VirtualBox ~/Desktop' with a menu bar (File, Edit, View, Search, Terminal, Help). The terminal shows the command 'autoexpect ./questions' being executed. The output is: 'autoexpect started, file is script.exp', 'Hello, who are you?', 'Adam', 'Can I ask you some questions?', 'Yes', 'What is your favorite topic?', 'Technology and programming', 'autoexpect done, file is script.exp'. The prompt returns to 'likegeeks@likegeeks-VirtualBox ~/Desktop \$' with a cursor.

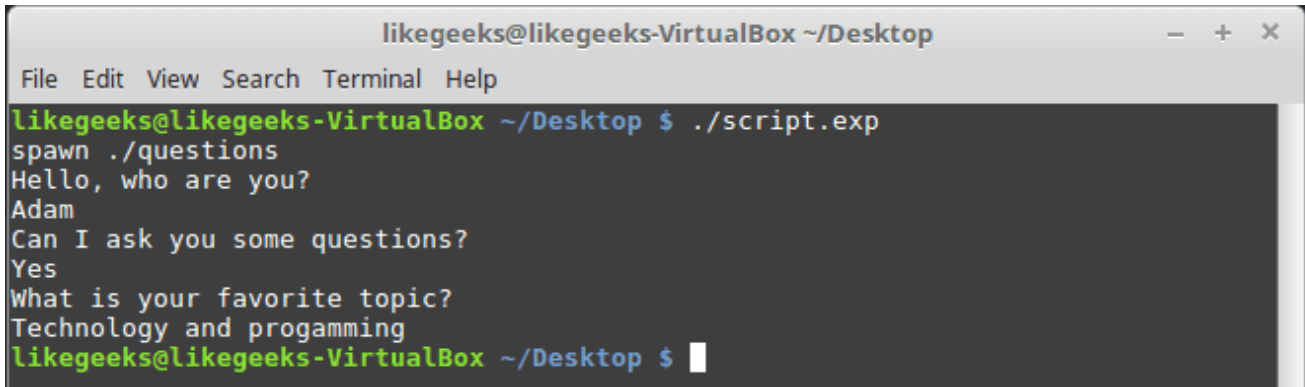
```
likegeeks@likegeeks-VirtualBox ~/Desktop
File Edit View Search Terminal Help
likegeeks@likegeeks-VirtualBox ~/Desktop $ autoexpect ./questions
autoexpect started, file is script.exp
Hello, who are you?
Adam
Can I ask you some questions?
Yes
What is your favorite topic?
Technology and programming
autoexpect done, file is script.exp
likegeeks@likegeeks-VirtualBox ~/Desktop $
```

The autoexpect generates a file called script.exp contains the same code as we did above with some additions that we will leave it for now.

A terminal window titled 'likegeeks@likegeeks-VirtualBox ~/Desktop' with a menu bar (File, Edit, View, Search, Terminal, Help). The window is running GNU nano 2.5.3 editing 'File: script.exp'. The content of the file is: '# -Don', 'set timeout -1', 'spawn ./questions', 'match_max 100000', 'expect -exact "Hello, who are you?\r"', 'send -- "Adam\r"', 'expect -exact "Adam\r"', 'Can I ask you some questions?\r', 'send -- "Yes\r"', 'expect -exact "Yes\r"', 'What is your favorite topic?\r', 'send -- "Technology and programming\r"', 'expect eof'. At the bottom, there is a status bar '[Read 59 lines]' and a keyboard shortcut menu with options like '^G Get Help', '^O Write Out', '^W Where Is', '^K Cut Text', '^J Justify', '^C Cur Pos', '^X Exit', '^R Read File', '^_ Replace', '^U Uncut Text', '^T To Spell', and '^_ Go To Line'.

```
likegeeks@likegeeks-VirtualBox ~/Desktop
File Edit View Search Terminal Help
GNU nano 2.5.3 File: script.exp
# -Don
set timeout -1
spawn ./questions
match_max 100000
expect -exact "Hello, who are you?\r"
send -- "Adam\r"
expect -exact "Adam\r"
Can I ask you some questions?\r
send -- "Yes\r"
expect -exact "Yes\r"
What is your favorite topic?\r
send -- "Technology and programming\r"
expect eof
[ Read 59 lines ]
^G Get Help ^O Write Out ^W Where Is ^K Cut Text ^J Justify ^C Cur Pos
^X Exit ^R Read File ^_ Replace ^U Uncut Text ^T To Spell ^_ Go To Line
```

If you run the auto-generated file script.exp, you will see the same answers as expected:

A screenshot of a terminal window titled 'likegeeks@likegeeks-VirtualBox ~/Desktop'. The window has a menu bar with 'File', 'Edit', 'View', 'Search', 'Terminal', and 'Help'. The terminal shows the following interaction: the user runs './script.exp', which outputs 'spawn ./questions', 'Hello, who are you?', 'Adam', 'Can I ask you some questions?', 'Yes', 'What is your favorite topic?', and 'Technology and progamming'. The prompt returns to 'likegeeks@likegeeks-VirtualBox ~/Desktop \$' with a cursor.

```
likegeeks@likegeeks-VirtualBox ~/Desktop
File Edit View Search Terminal Help
likegeeks@likegeeks-VirtualBox ~/Desktop $ ./script.exp
spawn ./questions
Hello, who are you?
Adam
Can I ask you some questions?
Yes
What is your favorite topic?
Technology and progamming
likegeeks@likegeeks-VirtualBox ~/Desktop $
```

Awesome!! That super easy.

Many commands produce changeable output, like the case of FTP programs, the expect script may fail or stuck. To solve this problem, you can use wildcards for the changeable data to make your script more flexible.

Working with Variables

You can use the set command to define variables in expect scripts like this:

```
set MYVAR 5
```

To access the variable, precede it with \$ like this \$VAR1

To define command line arguments in expect scripts, we use the following syntax:

```
set MYVAR [lindex $argv 0]
```

Here we define a variable MYVAR, which equals the first passed argument.

You can get the first and the second arguments and store them in variables like this:

```
set my_name [lindex $argv 0]

set my_favorite [lindex $argv 1]
```

Let's add variables to our script:

```
#!/usr/bin/expect -f

set my_name [lindex $argv 0]

set my_favorite [lindex $argv 1]

set timeout -1

spawn ./questions

expect "Hello, who are you?\r"

send -- "Im $my_name\r"

expect "Can I ask you some questions?\r"
```

```
send -- "Sure\r"

expect "What is your favorite topic?\r"

send -- "$my_favorite\r"

expect eof
```

Now try to run the Expect script with some parameters to see the output:

```
$ ./answerbot SomeName Programming
```

Awesome!! Now our automated Expect script is more dynamic.

Conditional Tests

You can write conditional tests using braces like this:

```
expect {  
  
    "something" { send -- "send this\r" }  
  
    "*another" { send -- "send another\r" }  
  
}
```

We are going to change our script to return different conditions, and we will change our Expect script to handle those conditions.

We are going to emulate different expects with the following script:

```
#!/bin/bash  
  
let number=$RANDOM  
  
if [ $number -gt 25000 ]; then  
  
    echo "What is your favorite topic?"  
  
else  
  
    echo "What is your favorite movie?"  
  
fi
```

```
read $REPLY
```

When you run the script, it generates a random number every time, and based on that number; we put a condition to return different expects.

Let's make out Expect script that will deal with that.

```
#!/usr/bin/expect -f

set timeout -1

spawn ./questions

expect {

    "*topic?" { send -- "Programming\r" }

    "*movie?" { send -- "Star wars\r" }

}

expect eof
```

Very clear. If the script hits the topic output, the expect script will send programming, and if the script hits movie output, the expect script will send star wars. Isn't cool?

If else Conditions

You can use if/else clauses in expect scripts like this:

```
#!/usr/bin/expect -f

set NUM 1

if { $NUM < 5 } {

    puts "\Smaller than 5\n"

} elseif { $NUM > 5 } {

    puts "\Bigger than 5\n"
```

```
} else {  
  
    puts "\Equals 5\n"  
  
}
```

Note: The opening brace must be on the same line.

While Loops

While loops in expect language must use braces to contain the expression like this:

```
#!/usr/bin/expect -f  
  
set NUM 0  
  
while { $NUM <= 5 } {  
  
    puts "\nNumber is $NUM"
```

```
set NUM [ expr $NUM + 1 ]

}

puts ""
```

For Loops

Like any scripting or programming language, you can use for loops for repetitive tasks, three fields must be specified, like the following format:

```
#!/usr/bin/expect -f

for {set NUM 0} {$NUM <= 5} {incr NUM} {
```

```
puts "\nNUM = $NUM"

}

puts ""
```

User-defined Functions

You can define a function using proc like this:

```
proc myfunc { TOTAL } {

    set TOTAL [expr $TOTAL + 1]

    return "$TOTAL"
```

```
}
```

And you can use them after that.

```
#!/usr/bin/expect -f

proc myfunc { TOTAL } {

    set TOTAL [expr $TOTAL + 1]

    return "$TOTAL"

}

set NUM 0

while {$NUM <= 5} {

    puts "\nNumber $NUM"

    set NUM [myfunc $NUM]

}
```

```
puts ""
```

Interact Command

Sometimes your expect script contains some sensitive information that you don't want to share with other users who use your expect scripts, like passwords or any other data. Hence, you want your script to take this password from you and continuing automation normally.

The interact command reverts the control to the keyboard.

When you use the interact command, expect will start reading from the keyboard.

This shell script will ask about the password as shown:

```
#!/bin/bash

echo "Hello, who are you?"

read $REPLY

echo "What is your password?"

read $REPLY

echo "What is your favorite topic?"

read $REPLY
```

Now we will write the Expect script that will prompt for the password:

```
#!/usr/bin/expect -f

set timeout -1

spawn ./questions

expect "Hello, who are you?\r"

send -- "Hi Im Adam\r"
```

```
expect "*password?\r"

interact ++ return

send "\r"

expect "*topic?\r"

send -- "Technology\r"

expect eof
```

After you type your password type ++ and the control will return from the keyboard to the script.

Expect language is ported to many languages like C#, Java, Perl, Python, Ruby, and Shell with almost the same concepts and syntax due to its simplicity and importance.

You can use the expect scripting language in quality assurance, network measurements such as echo response time, automate file transfers,

updates, and many other uses.

I hope you now supercharged with some of the most important aspects of expect command, autoexpect command, and how to use it to automate your tasks in a smarter way.

Thank you.

[Share on Facebook](#)[Tweet on Twitter](#)

Mokhtar Ebrahim

Mokhtar is the founder of LikeGeeks.com. He is a seasoned technologist and accomplished author, with expertise in Linux system administration and Python development. Since 2010, Mokhtar has built an impressive career, transitioning from system administration to Python development in 2015. His work spans large corporations to freelance clients around the globe. Alongside his technical work, Mokhtar has authored some insightful books in his field. Known for his innovative solutions, meticulous attention to detail, and high-quality work, Mokhtar continually seeks new challenges within the dynamic field of technology.



53 thoughts on “Expect command and how to automate shell scripts like magic”



joydeep das says:

2018-08-27 at 11:07 am

That a very well written article 😊

[Reply](#)



Mokhtar Ebrahim says: